TeamForge

Instructor Guide

How to build a survey, collect anonymous responses, and let the optimizer form balanced student teams — privately, end to end.

What TeamForge does

TeamForge helps you sort a class into project teams while honoring rules you set — team sizes, skill balance, diversity safeguards, project requirements, and student teammate preferences. Students answer a short survey anonymously; a mixed-integer optimizer then proposes an assignment you can fine-tune by hand.

Privacy in one line

Student answers are encrypted in the student's browser before upload, the decryption key never reaches the server, and the optimizer runs inside your browser. The platform stores only ciphertext and never receives student names or emails.

This guide follows one running example — "MGMT 4500, Spring 2027", 30 students, six project teams — from sign-up to final roster.

1. The big picture

A session moves through these stages. You can revisit earlier stages until you open the survey.

1. Register and get approved — sign up with your university email; an administrator approves your account.
2. Create a session — set student count, team sizes, and an encryption passphrase. You download two files once. The count is a starting point, not a commitment: students can be added and removed later on the Roster tab.
3. Define projects (optional) — describe each project and any attribute requirements (e.g. “needs >=1 CS major”).
4. Build the survey — add demographic, skill, and preference questions, often from built-in standard scales.
5. Set constraints — choose which rules matter and how strongly (Must / Important / Nice to have).
6. Open the survey — students log in with a private code, see a share code, and submit encrypted answers.
7. Close and allocate — unlock with your passphrase, run the optimizer, adjust teams, export the roster.
8. Team management (optional) — keep the session going after allocation to run team contracts and two rounds of peer evaluations (see §11).
9. Erase — purge student data once teams are final.

Tabs you will use

Inside a session: Overview (status, invite text, delete) · Projects · Survey · Constraints · Roster (completion, adding and removing students) · Allocation (optimize & export) · Privacy & data (erase). A “Projects” tab appears only for sessions with named projects; “Teams” and “Peer evals” tabs appear once you enable the optional team-management phase (§11).

2. Getting an account

- Click Register and enter your name, your university/institution, and your email. Please use your university email — it speeds up approval.
- Verify your email via the link sent to you, then reload.
- Your account then waits for manual approval by the site administrator. You cannot create sessions until approved.
- Once approved, sign in to reach My sessions (your dashboard).

Why approval?

Approval keeps the free service limited to legitimate instructors. The administrator only sees your name, university, and email — never any student data.

3. Creating a session

From My sessions, click New session. For our example we enter:

- Title: “MGMT 4500 — Operations Strategy, Spring 2027”.
- Students: 30.
- Ideal team size: 5. Min and Max auto-fill to 4 and 6 (ideal ± 1); you can override them.
- Generic projects: left unchecked — we want six named projects, one team each. (Check it instead to just split the class into N teams with no project descriptions.)
- Encryption passphrase: a strong phrase of at least 10 characters, entered twice.

When you click Create, TeamForge generates the session and downloads two files. This is the only time you receive them.

File 1 — student codes (CSV)

- One row per student with: studentIndex, loginCode, shareCode, surveyLink, and blank columns for the student's name and email.
- The loginCode is private to each student — it is their key to the survey and is never stored on the server (only a hash is).
- The shareCode is a short public code students use to name preferred teammates. It is safe to share and cannot be used to log in.
- Fill in the name/email columns yourself to keep your private mapping; this file never leaves your computer.

File 2 — recovery key

A text file containing a recovery key that can unlock student data if you forget your passphrase. Store it somewhere safe and separate from the codes.

> **Read this twice**
>
> The passphrase and recovery key are the ONLY ways to decrypt student answers. If you lose both, the data is permanently unreadable — by design, no one (including the platform operator) can recover it for you.

4. Defining projects

On the Projects tab, add each project with a name and a description students will see. Optionally add requirements — attributes a team must contain.

In our example we add six projects. For the “Data Dashboard” project we add a requirement:

- Attribute: “Major”. Because Major is a recognized standard scale, the Required value becomes a dropdown — we pick “Computer Science” (choosing from the list avoids near-duplicate categories like “CS” vs “Computer Science”).
- Min count: 1 — the team needs at least one Computer Science major.

Requirements wire themselves into the survey

Each requirement automatically adds a matching question to the survey (e.g. a “What is your Major?” single-choice question, pre-filled with the standard list of majors) and an umbrella “project requirements” constraint you can weight on the Constraints tab. Edit the generated question's options freely afterward.

5. Building the survey

On the Survey tab, click Add question. The fastest path is Start from a standard scale, a dropdown of vetted questions grouped into Demographics, Academic, Skills & ratings, and Work style. Selecting one fills in the type, prompt, and options, which you can then edit.

Question types

- Single choice / Multiple choice — categorical options, one per line. Used for demographics, major, roles, etc.
- Numeric scale — a 1–5 (or any range) rating. You can attach a word to each number (e.g. 1 = “No experience” … 5 = “Expert”). Numbers are what the optimizer balances; the words are shown to students for clarity.
- Preferred teammates — students enter the share codes of classmates they want to work with (see §7).

For MGMT 4500 we add, from the standard scales:

- Gender (Demographics) — to keep teams from isolating a single student of one gender.
- Proficiency in spreadsheet modeling (Skills & ratings, numeric 1–5 with labels) — to balance skill across teams.
- Preferred teammates (Work style) — so friends can request each other.
- Plus the auto-generated “What is your Major?” question from the Data Dashboard requirement.

Reusing a survey from another session

If you have run TeamForge before, the Survey tab offers “Reuse another session's survey”. Pick one of your own sessions and TeamForge copies its questions together with the constraints that reference them — both at once, because a constraint names the question it works on. Everything is added to what this session already has; nothing is replaced.

Before it writes anything you see exactly what will be added, and what will not. Two things never come across: questions marked “auto”, which are rebuilt from this session's own project requirements, and any constraint whose question therefore did not arrive. The reasons are listed so nothing goes missing quietly.

> **No free-text answers — by design**
>
> Students can only pick options or numbers; there is no place to type a name. This is one reason the platform never learns who anyone is.

6. Setting constraints

Constraints are the rules the optimizer trades off. Team-size limits are always enforced; everything else carries a weight: Must hold, Important, or Nice to have. Higher weight = the optimizer works harder to satisfy it.

Constraints you add yourself

- Anti-isolation — never put exactly one student with a given value (e.g. exactly one woman) on a team: each team gets zero or at least two. Great for our Gender question.
- Category coverage — every team needs at least N students who gave a particular answer (e.g. >=1 student answering “I prefer to lead” on the leadership question, so no team is left without anyone willing to coordinate). Works on multi-select questions too, so “Leader / coordinator” among preferred roles counts.
- Capability coverage — every team needs at least N students scoring >= a threshold on a numeric question (e.g. >=1 student rating spreadsheet skill >=4).
- Alignment — team members should give the same answer to a single-choice question (e.g. mostly in person / hybrid / mostly remote), so a team shares a way of working instead of splitting over it. Each team settles on whichever answer most of its members gave; every member outside it costs the weight once. The optimizer picks the answer per team, so no way of working is privileged.
- Balance a numeric attribute — spread a numeric question's average evenly across teams (e.g. even spreadsheet skill).

Coverage and alignment pull in opposite directions on purpose: coverage spreads a scarce answer across teams, alignment concentrates one inside each team. Anti-isolation is a third thing again — it asks for none or at least two, never exactly one.

Constraints TeamForge manages or suggests for you

- Project requirements (umbrella) — appears automatically when projects have requirements, tagged “from Projects”. Set how strongly to enforce it with its weight; edit the underlying requirements on the Projects tab.
- Suggested constraints — when your survey has a teammates or project-ranking question, a one-click suggestion appears to add “Respect teammate preferences” or “Respect project preferences”. Add it, then adjust its weight.

For our example we set: project requirements = Must hold; anti-isolation on Gender = Important; balance spreadsheet skill = Important; at least one student willing to lead per team = Important; alignment on work preference = Nice to have; respect teammate preferences = Nice to have.

7. Opening the survey and inviting students

- On the Overview tab, set the status to Open. (Draft = students can't submit yet; Closed = submissions locked.)
- Copy the survey link and the editable email template. Mail-merge a message to each student with their personal loginCode from the CSV.
- Edit and Save the email template per course; placeholders like <LOGIN CODE> are filled per student by your mail-merge.

Login codes vs. share codes — keep them straight

- Login code (private): each student's key to open and submit the survey. Never shared with classmates.
- Share code (public): shown to the student after they log in. They exchange these with friends and type them into the “preferred teammates” question. A share code cannot be used to log in, so sharing it is safe.

What the student sees

The student opens the link, reads a privacy notice, enters their login code, and (if a teammates question exists) sees their own share code to pass to friends. Answers are encrypted in their browser on submit. They may edit or withdraw their response any time while the survey is open.

8. The roster: tracking responses, adding and removing students

The Roster tab shows completion by student index (never by name — the platform doesn't have names). Use it to see who still needs a reminder. You cannot read individual answers here; they stay encrypted until you unlock them on the Allocation tab.

Students who join or drop after you created the session

Enrollment moves for the first weeks of term, and teams have to be assigned before it settles. You do not have to guess the final number when you create the session, and you do not have to start over when it changes.

- Add students: choose how many and press Add. You get a small CSV of their login codes, in the same columns as the file you downloaded at creation. Append those rows to your master copy — the codes are shown once here too, and cannot be recovered afterwards.
- Remove students: select their tiles and press Remove. Their record and any survey response are deleted, and their login code stops working immediately.
- Student numbers are never reused. If you remove #14 and later add someone, they get the next unused number, not 14. The number is what identifies a student inside the encrypted team rosters and peer evaluations, so reusing it would silently attach the newcomer to the previous holder's history.

What to redo after a roster change

Anything already worked out from the roster is now out of date, and TeamForge says so rather than quietly using it. If you had saved an allocation, re-run the optimizer and save. If you had provisioned teams, the Teams tab shows a banner with an Update teams now button: unlock and press it, and a departed student is taken off their teammates' contract and peer-evaluation form, with no file to upload. A student you added here is on no team until you place them on the Teams tab (see section 11) — or add them from the Teams tab in the first place, which does both at once.

9. Closing and allocating teams

- Set the session status to Closed so no further submissions arrive.
- Open the Allocation tab and unlock with your passphrase (or the recovery key). Decryption happens only in your browser.
- Optionally set a solver time limit (seconds), then click Run optimizer. The optimizer runs locally as WebAssembly.
- Review the proposed teams on the board. Each team shows which constraints it satisfies or violates.
- Drag students between teams to adjust by hand; violation feedback updates live.
- Click Save (encrypted) to store the allocation (encrypted), and Export CSV to download the final roster (team + student index).

Large classes

The model uses one variable per (student × team). Very large instances may not reach a proven optimum within the time limit — raise the limit, accept a good-enough result, or use fewer teams. TeamForge warns you when a problem is large.

10. Privacy, data, and erasure

What is guaranteed

- Students are identified only by random codes; the platform never receives names or emails.
- Answers are encrypted in the student's browser; the server stores only ciphertext.
- The decryption key (passphrase / recovery key) is never sent to the server; only you can decrypt.
- Optimization runs in your browser; the saved allocation is encrypted too.
- No analytics, tracking, or third-party scripts on student pages.

The honest caveat

Like all browser-delivered encryption, these guarantees assume the app's code is served honestly. An operator who tampered with the deployed app could capture data before it is encrypted. The protections fully cover anyone reading the stored database, which is the realistic threat.

Erasing data

- Purge student data (Privacy & data tab) deletes all encrypted responses and the saved allocation, keeping the session shell.
- Delete session (Overview or Privacy & data tab) removes everything for the session.
- Good practice: purge once teams are final, and delete the private codes CSV from your computer at the end of term.

11. Team management (optional)

After you close a session and allocate teams, you can optionally keep using it to run team contracts and peer evaluations. This is entirely optional — if you only need team allocation, ignore this chapter. Enable it from the Overview tab once the session is Closed; two new tabs appear, Teams and Peer evals.

Uploading the login-codes CSV

- On the Teams tab, upload the student-codes.csv you downloaded when you created the session — exactly as it is. You do not add a team column: TeamForge reads the teams from the allocation you already saved, matching each row by its login code. Unlock with your passphrase when asked, since the allocation is encrypted.
- Why upload anything at all? Login codes are shown to you once and never stored — the platform keeps only their hashes. Each student's team is sealed under a key derived from their own code, so that code has to come from you. The teams do not: those the app already has.
- Adjusted teams by hand after allocating, or never ran the optimizer? Add a team column (any label, e.g. "Team 1" or a project name). Any row that has one overrides the saved allocation; rows left blank still come from it, so you only need to fill in the students you moved.
- You do not upload names. If your working sheet already has a name column, leave it — TeamForge shows those names in the on-screen preview so you can confirm you picked the right file, but they never leave your browser.
- Check the preview before provisioning. It reports how many assignments came from the allocation versus your file, and warns if anyone in the allocation is missing from the CSV — which is what a truncated or stale file looks like.
- Your browser then encrypts each student's team membership under their own key, so the platform never stores team membership in plaintext either. Students log in with the same code they used for the survey.
- Once teams are provisioned, the Teams tab offers a ready-made email announcing the team and the contract, and the Peer evals tab offers one per round. Each is editable, saved with the session, and uses the same <STUDENT NAME> / <LOGIN CODE> / <DEADLINE> placeholders as the survey email, so one mail merge covers the whole term. The graded-round draft quotes this session's own factor caps and links the worked-example workbook, so what students are told matches what the app computes.

Changing teams after they are provisioned

Students add and drop for weeks after teams are set. Once teams are provisioned, you change them one student at a time on the Teams tab, after unlocking with your passphrase. None of this needs the codes CSV: the encrypted team directory already holds what is needed to re-seal each provisioned student's team.

- Add a student: press Add a student (or Add a student to this team inside a team). For someone new to the class, a login code is minted, downloaded as a one-row CSV exactly as on the Roster tab, and they are placed on the team in the same step. Keep those rows with your master codes file. For someone who already has a code but is on no team — added on the Roster tab after provisioning — type their code; it is used once, in your browser, to derive their key.
- Move a student: open their team, press Move next to their name and pick the destination, or type a name to create a new team (a name that matches an existing team joins it). Both teams' members see the change on their next login. Display names are sealed per team, so the moved student picks theirs again; their old team's contract stays as it was, without them.
- Remove a student who dropped: press Remove from session next to their name. This is the

same removal as on the Roster tab — their record and response are deleted, their code stops working, their number is retired — and their teammates stop seeing them straight away.
- Delete a team that has emptied: a team with nobody left on it is kept, contract and all, until you press Delete team inside it.
- Mid-round moves: if a peer-evaluation round is open when you move somebody, the ballot they already submitted was about their old teammates, so it is withdrawn and they are asked to fill one in for the new team. Ballots their old teammates submitted that named them are still accepted; the points allocated to them are dropped and the rest scaled back up, as for a student who left. Teammates on either team who had already submitted did not rate them. Where you can, move students between rounds rather than during one.
- The whole-roster re-upload remains for wholesale changes. It now starts from the teams as they stand, so students you moved by hand are not moved back, and a team column in the file still wins.

Display names (students choose their own)

- On first login a student is asked to choose a display name. It is encrypted under their team's key, so only their teammates and you can read it — the platform stores it unreadable.
- Students may use their real name, a short form, or any nickname; the app tells them to share their choice with their teammates so everyone knows who is who when evaluating.
- Until a student chooses one, they appear to teammates and to you as their code number (e.g. #7). Everything still works — the code index, not the name, is what the factor maths uses.
- Your review tables and CSV exports show the display name next to the code index. Because you hold the codes CSV, you can map a code index back to the real student for your gradebook; TeamForge itself never learns that mapping.

Team contracts

- Any one team member drafts the contract (communication, attendance, timeliness, respect, effort, integrity, plus custom sections). All members can view and edit it.
- Teams may optionally request AI feedback, revise, and finalize. Every member can save the finalized contract as a PDF.
- You can read every team's contract on the Teams tab after unlocking with your passphrase, and download them all.

Peer evaluations

- Two rounds: a practice (formative) round whose results you can return privately, and a graded (summative) round. Open and close each round on the Peer evals tab.
- The form is fixed: allocate 100 points across teammates (an equal split is the default and the neutral answer; any allocation far enough from it to actually move that teammate's factor needs a one-sentence justification — on a team of five that means anything outside 23–27), rate four behaviors 1–5 (required unless you untick “Include behavior ratings” in the Peer evals settings), and an optional confidential comment to you.
- Watch completion by code number, then unlock to compute each student's team factor. The factor multiplies the team-scored part of a grade. Everything is computed in shares, where 1.00 is an even split: your share of one ballot is the points you got divided by 100 ÷ (team size - 1). The highest and the lowest share you received are dropped, the rest averaged, and the result mapped through a dead band (delta), damping (k) and asymmetric caps — f = clip(1 + k * sign(d) * max(0, |d| - delta), floor, ceiling), where d is your average share minus 1. Defaults: delta = 0.08, k = 0.5, floor 0.70, ceiling 1.05.
- A teammate who does not submit is treated as having split evenly, so skipping the form neither helps them nor penalises everyone else.
- Two deliberate asymmetries. Dropping both the highest and the lowest share means one hostile

rater and one over-generous rater are equally powerless. And the ceiling sits closer to 1.00 than the floor does, so a group that agrees to sink one member gains far less than the target loses — the arithmetic makes that play cost the team rather than pay it.

- Factors below 0.90, teams whose factors spread by more than 0.20, and members everyone rated the same and low are flagged for your attention before any grade is issued. Export summary and detail CSVs, and optionally publish each student's own factor back to them privately.
- A ready-made worked example ships with the app at /peer-eval-team-factor.xlsx — the same five-member team, as a live Excel calculator rather than a picture of one. Every figure below the ballots is a formula, so students can change a number and watch the result move. There is a download link on the Peer evals tab.
- The table shows a team mean. It reads exactly 1.00 for any team without real dispersion and dips below only when someone genuinely under-contributed — the dead band is what makes that true, and nothing rescales the numbers behind your back.

AI feedback is optional and clearly bounded

AI contract feedback runs through a small proxy you deploy (see worker/README.md). It is the one point where contract text — never names — leaves the end-to-end encryption. If you don't configure it, the feature is simply hidden and everything else works. Students see an explicit disclosure and must confirm before any text is sent.

12. Troubleshooting & FAQ

“Awaiting approval” after registering.
Your email is verified but an administrator hasn't approved you yet. You can't create sessions until then.

I forgot my passphrase.
Unlock with the recovery key file instead (Allocation tab). Then consider re-creating the session with a passphrase you'll remember.

I lost both the passphrase and recovery key.
The student answers are permanently unreadable — by design. Re-open the survey with new codes if you must collect again.

A student joined the class in week 2.
Before teams exist: Roster tab, Add. You get their login code in a small CSV — append those rows to your master codes file — and re-running the optimizer is what gives them a team. After teams are provisioned: Teams tab, Add a student, pick their team. Their code is minted and downloaded and they are placed in one step, with nothing to re-upload.

A student dropped the class.
Roster tab: select their tile and Remove — or, once teams are provisioned, open their team on the Teams tab and press Remove from session, which takes them off the team at the same time. Either way their record and response are deleted and their code stops working. If you removed them on the Roster tab, the Teams tab shows a banner: press Update teams now so their teammates stop seeing them on the contract and the peer-evaluation form. Their number is retired and will not be handed to anyone else.

A student needs to switch teams.
Teams tab: open their current team, press Move next to their name and choose the destination — or type a name to start a new team. Their contract access and peer-evaluation form follow them on their next login; they choose their display name again on the new team. If a round is open, the ballot they already submitted is withdrawn so they can fill one in for the new team.

I removed a student mid-way through a peer-evaluation round. Did I ruin it?
No. Ballots already submitted are kept: the points allocated to the departed student are dropped and the rest scaled back up to 100, so what each rater said about the teammates who remain is preserved exactly. The same happens when you move a student to another team mid-round. The review screen lists which ballots were adjusted, and the detail CSV shows both what was submitted and what was scored.

Can a student I removed still get at their old team's contract?
Possibly, and it is worth knowing. Their login stops working at once, but a team's documents are reached by holding an unguessable token, and if they had already logged in they may still hold it. To cut that off, re-upload the roster giving that team a different label — that mints a new key. It also starts that team's contract and display names over, so it is a deliberate trade rather than something to do routinely.

A student says their code isn't recognized.
Check for typos; codes are case-insensitive and tolerant of dashes/spaces. Make sure they're using their login code, not a share code, and that the survey is Open.

The same category appears twice (e.g. “Woman” and “Female”).

Use the standard-scale dropdown for both the survey option and the project requirement value so they match exactly; remove the stray option from the survey question.

The optimizer won't finish / says the problem is large.

Raise the time limit, reduce the number of teams, or relax some Must constraints to Important.

Nothing happens when I change something.

TeamForge shows an inline error if a save fails (e.g. you're offline). Check your connection and try again.

The AI feedback button doesn't appear for teams.

It only shows when the AI proxy is configured (VITE_AI_PROXY_URL) and "Offer AI contract feedback" is checked in the Peer evals settings. Without the proxy, contracts still work — just without AI feedback.

A peer-eval factor looks off.

Check the Share column first: 1.00 is an even split, and anything within the dead band (±0.08 by default) maps to a factor of exactly 1.00. The highest and lowest share a student received are dropped before averaging, and the Trimmed column shows which two went. Export the detail CSV to see every allocation.

The team mean is below 1.00. Is that a bug?

No. The dead band and the deliberately tight ceiling mean the factors are not a fixed pot being redistributed. A team where everyone contributed evenly averages exactly 1.00; a team carrying a free rider averages less, which is the honest reading. Nothing is rescaled to hide it.

A group could agree to give one member nothing and split the rest. What stops them?

Less than you might fear, and the numbers say so plainly. Four members dumping on a fifth gain 0.05 each while the target loses 0.30, so the play costs the team more than it pays the plotters — and each of them must write a justification for every allocation involved. Watch for the “unanimous low” flag: independent raters sizing up a real free rider disagree with each other, while people working from an agreed number do not. Then read their justifications in the detail CSV; near-identical wording is the tell. Treat it as a prompt to talk to the team, never as proof.

A student shows as "#7" instead of a name.

They haven't chosen a display name yet — it appears as soon as they log in and pick one. Nothing is blocked in the meantime: contracts and peer evaluations work, and the code index is what the factor calculation uses either way.

Two students picked confusingly similar display names.

The app already refuses an exact duplicate within a team. For near-duplicates, ask one of them to change it (Change on their hub); tables and exports also show the code index so you can always tell them apart.

How do I get from a display name back to the real student?

Use the code index shown beside it and your own login-codes CSV, where you kept the code-to-student mapping. That join happens in your spreadsheet — the platform never holds it.